Facilitator & Settlement Mesh for x402 on Algorand

Final Implementation Report — Team Agni, Algorand Sponsor Track

Abstract

x402 revives HTTP 402 to let resource servers charge per request and lets clients — including autonomous AI agents — pay in-band with a signed transaction. The protocol is intentionally chain-agnostic and leaves settlement correctness entirely to whoever implements the facilitator. Published critiques of x402 deployments (A402, 2026) identify three structural weaknesses in typical EVM-based facilitator implementations: settlement latency bounded by confirmation time, fee cost that makes fine-grained micropayments uneconomical, and lack of enforced end-to-end atomicity between payment and receipt. This report specifies a non-custodial Facilitator & Settlement Mesh for x402's `exact` scheme on Algorand (AVM), built as seven cooperating modules, each directly answering one of the problem statement's required capabilities. Rather than claiming Algorand is categorically faster or cheaper than every alternative — Base and Solana already offer sub-cent fees and fast soft-confirmation — this report argues Algorand's specific advantage for a settlement mesh is atomic composability without a deployed contract: a payment and its cryptographic receipt can be forced to succeed or fail together, natively, in one signed group, with deterministic (not probabilistic) finality in under three seconds.

1. Problem Statement Compliance Map

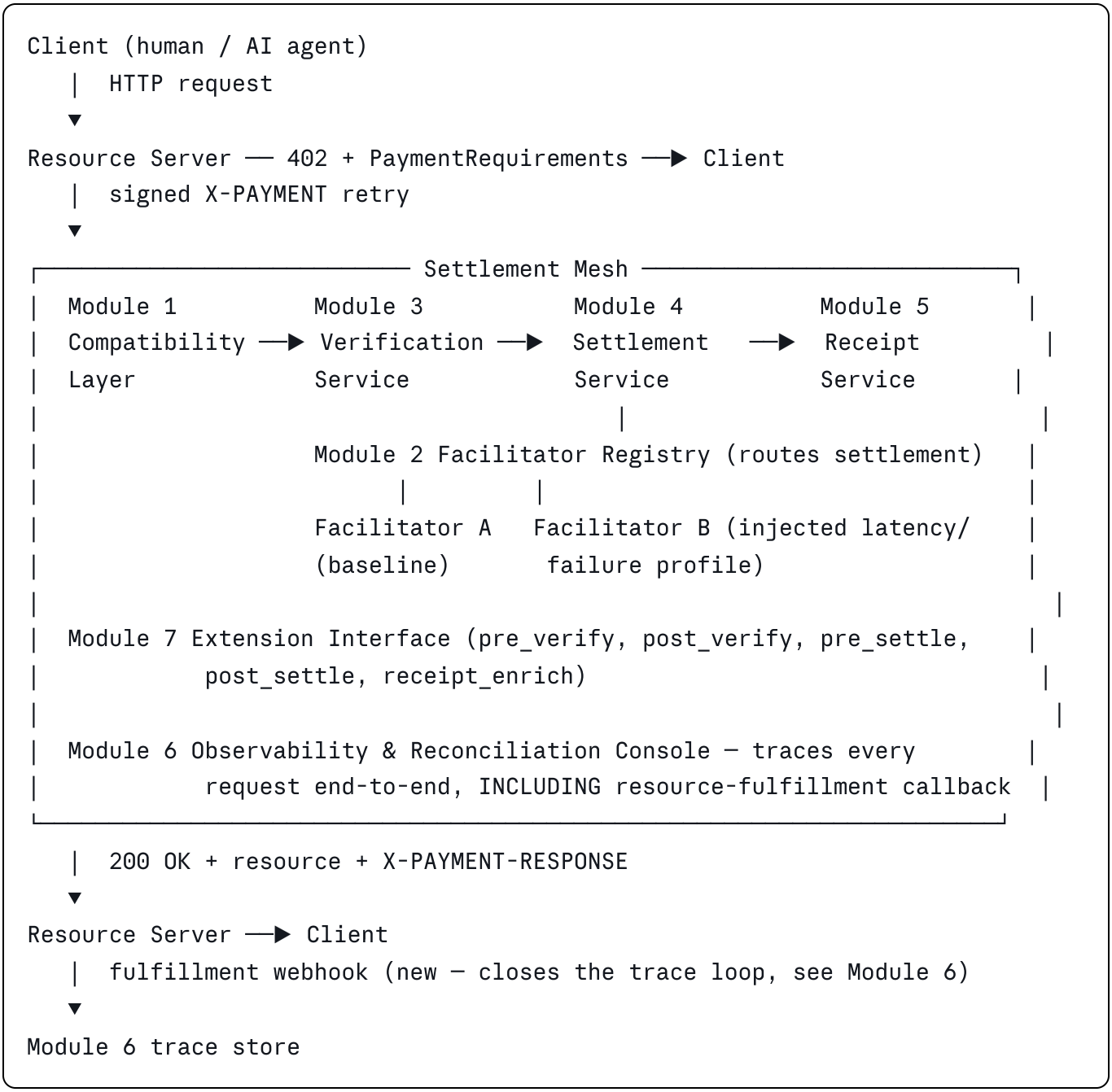
PS-required capability	Module
Compatibility layer (schemes, networks, versioning)	Module 1
Facilitator registry (discovery, health, routing, circuit breakers)	Module 2
Verification service	Module 3
Settlement service (idempotency, state machine, confirmation tracking)	Module 4
Receipt service	Module 5
Observability / dispute-reconciliation console (full trace incl. fulfillment)	Module 6
Extension / plugin interface	Module 7

2. Research Grounding

- **x402 specification** (`x402-foundation/x402`, formerly `coinbase/x402`) — defines the protocol as scheme/network-pair-based; requires facilitators to explicitly declare supported `(scheme, network)` pairs; states trust-minimization as a core design goal — no scheme may let a facilitator or resource server move funds outside client intent.
- **A402** (Li et al., 2026) — identifies x402's three structural gaps (latency, fee cost, atomicity). This report closes the atomicity and latency gaps at the settlement layer using native Algorand primitives rather than an off-chain channel.
- **SoK: Blockchain Agent-to-Agent Payments** (2026) — systematizes the facilitator-trust problem: a resource server cannot verify a facilitator's settlement claim without re-querying the chain itself. Module 5's independently-signed receipts and Module 6's audit trail respond to this directly.
- **Gilad, Hemo, Micali, Vlachos, Zeldovich — "Algorand: Scaling Byzantine Agreements for Cryptocurrencies"** (SOSP '17) — formal basis for treating Algorand block finality as deterministic, which is what lets Module 4's state machine omit reorg-handling states.
- **Rivest & Shamir — "PayWord and MicroMint"** (1996) — conceptual ancestor of Module 4's idempotency vault: a broker-side record of already-spent tokens, here an atomically-claimed key rather than a hash chain.
- **HexHive** `x402scope` — an existing security-testing tool for EVM/Solana x402 facilitators. No AVM-targeted or protocol-official conformance suite was found publicly (see §10 for how this report handles that gap honestly rather than silently).

---

3. System Architecture



Module 1 — Compatibility Layer

python

```
SUPPORTED_PAIRS = {
    ("exact", "algorand:localnet"),
}

def compatibility_check(payload: dict) -> CompatResult:
    scheme, network = payload["scheme"], payload["network"]
    if (scheme, network) not in SUPPORTED_PAIRS:
        return CompatResult(ok=False, reason="unsupported_scheme_network_pair")
    if payload.get("x402Version") != SUPPORTED_VERSION:
        return CompatResult(ok=False, reason="unsupported_protocol_version")
    return CompatResult(ok=True)
```

Failure mode closed: provider drift. Declaring `SUPPORTED_PAIRS` as an exhaustive, checked set — rather than assuming and failing deep in the pipeline — is what makes §10's conformance matrix honest rather than aspirational.

Module 2 — Facilitator Registry & Routing Mesh

python

```
class FacilitatorEntry:
    id: str
    endpoint: str
    supported_pairs: set[tuple[str, str]]
    health: Literal["healthy", "degraded", "down"]
    p50_latency_ms: float
    p99_latency_ms: float
    error_rate_5m: float
    circuit_state: Literal["closed", "open", "half_open"]
    consecutive_failures: int
```

Circuit breaker:

```
closed --[5 consecutive failures]--> open
open --[cool-down 30s elapsed]--> half_open
half_open --[next call succeeds]--> closed
half_open --[next call fails]--> open
```

Routing rule: select the healthy (`closed`/trial `half_open`) facilitator supporting the requested `(scheme, network)` pair with lowest `p50_latency_ms`; if none healthy, return `503 all_facilitators_unavailable` rather than retry indefinitely — "fail closed on ambiguity" applied at the routing layer.

Two-facilitator demo setup: Facilitator A runs against LocalNet with no injected delay; Facilitator B wraps the same LocalNet connection with a configurable latency/failure injector (e.g. 20% forced 500s, +500ms latency). This satisfies the PS's two-facilitator MVD, the facilitator-disagreement hard-mode extension, and circuit-breaker demonstration with one piece of infrastructure.

Module 3 — Verification Service

python

```python
async def verify(payload: dict) -> VerifyResult:
    if not compatibility_check(payload).ok:
        return VerifyResult(valid=False, reason="incompatible")
    if not verify_avm_signature(payload):
        return VerifyResult(valid=False, reason="bad_signature")
    if payload["amount"] != requirements.amount or payload["asset"] != requirem
        return VerifyResult(valid=False, reason="amount_mismatch")
    if payload["payTo"] != requirements.pay_to:
        return VerifyResult(valid=False, reason="recipient_mismatch")
    if payload["resource"] != requirements.resource:
        return VerifyResult(valid=False, reason="resource_mismatch")
    if is_expired(payload):
        return VerifyResult(valid=False, reason="payload_expired")
    if not claim_nonce(payload["nonce"]):
        return VerifyResult(valid=False, reason="nonce_reused")
    return VerifyResult(valid=True)
```

Failure modes closed: payload manipulation, malformed requirements, expired payload, network mislabeling.

Module 4 — Settlement Service (State Machine, Two-Layer Idempotency, Confirmation Tracking)

Layer A — request-level idempotency key (exists before any chain interaction):

python

```
idem_key = sha256(f"{payer}:{payTo}:{resource}:{amount}:{nonce}").hexdigest()
claimed = redis.set(f"idem:{idem_key}", "IN_PROGRESS", nx=True, ex=300)
if not claimed:
    cached = redis.get(f"idem:{idem_key}")
    if cached == b"IN_PROGRESS":
        return 409
    return json.loads(cached)
```

Layer B — TxID-level replay lock (once a signed transaction exists, pre-broadcast):

python

```
claimed = redis.set(f"txid:{txid}", "PENDING", nx=True, ex=86400)
if not claimed:
    existing = redis.get(f"txid:{txid}")
    if existing == b"SETTLED":
        return existing_receipt(txid)
    return 409
```

Confirmation tracking — the concrete mechanism (previously unspecified, now closed):

python

```
async def track_confirmation(txid: str, submitted_round: int, timeout_s: int =
    deadline = time.monotonic() + timeout_s
    while time.monotonic() < deadline:
        info = await algod_client.pending_transaction_info(txid)
        if info.get("confirmed-round", 0) > 0:
            return ConfirmResult(confirmed=True, round=info["confirmed-round"])
        if info.get("pool-error"):
            return ConfirmResult(confirmed=False, error=info["pool-error"])
        await asyncio.sleep(0.5)   # Algorand round time ~2.9s; poll well under
    return ConfirmResult(confirmed=False, error="timeout")
```

This polls `algod`'s `pending_transaction_info` directly rather than the indexer, because the indexer lags block finality by design (it's built for historical query, not real-time confirmation) — `algod` gives the confirmation the instant the block is certified. A 0.5s poll interval against a ~2.9s round time means confirmation is detected within one round in the common case, without hammering the node.

State machine:

```
RECEIVED → IDEMPOTENCY_CLAIMED → VERIFIED → ROUTED → SETTLING
   → CONFIRMED → RECEIPT_ISSUED     (success path, via track_confirmation
above)
   → FAILED                         (verification rejection or pool-error)
   → TIMEOUT                        (track_confirmation exceeded timeout_s)
```

Timeout handling: on `TIMEOUT`, the Layer A idempotency key remains claimed — a resource-server retry against the same `idem_key` re-enters `track_confirmation` for the same `txid` rather than re-settling, so a slow-but-eventually-successful transaction is still captured correctly instead of double-settled.

Why Algorand simplifies this specifically: per Gilad et al., certified blocks are not reorganized — `CONFIRMED` is a true terminal state, not a probabilistic one requiring `REORG_DETECTED`/`RECONFIRMING` states that a Nakamoto-consensus chain's settlement service would need.

Module 5 — Receipt Service

On-chain compact receipt (Note field, atomically grouped with the transfer):

json

```
{"v":1,"idem":"<idem_key first 16 hex>","txid":"<algo-txid>","status":"settled"
```

python

```
group = [payer_to_payee_transfer_txn, zero_algo_note_receipt_txn]
assign_group_id(group)   # atomic — both succeed or neither does
```

Off-chain full receipt, `GET /receipts/{idem_key}`, signed with a dedicated Ed25519 facilitator signing key held separately from any funds-holding key — so a second, unrelated service can verify the signature against the facilitator's published public key without trusting the dashboard UI at all, and a compromised API cannot forge a receipt for funds it never custodied.

Module 6 — Observability & Reconciliation Console (Trace Now Reaches Fulfillment)

Trace store:

python

```
class TraceEvent:
    idem_key: str
    timestamp: datetime
    module: str      # "compatibility" | "verification" | "settlement" | "recei
    state: str
    detail: dict
```

Closing the fulfillment gap: the mesh's process boundary ends at `RECEIPT_ISSUED` — it doesn't itself serve the paid resource. To satisfy the PS's explicit requirement to trace "through resource fulfillment," the resource server is given a fulfillment webhook to call back into the mesh:

```
POST /trace/{idem_key}/fulfillment
{ "fulfilled": true, "served_at": "<timestamp>" }
```

This is optional at the protocol level (a resource server that doesn't call it simply leaves that leg of the trace empty, which the console displays explicitly as `fulfillment: not_reported` rather than silently omitting it) but is wired into the demo's own resource server so the full 402 → verify → settle → receipt → fulfillment → final-response chain is genuinely visible in one console query, not reconstructed after the fact.

Console views: live chaos view, trace-by-`idem_key`/`txid` view, and `GET /audit/export?from=...&to=...` for the named audit-export deliverable.

Module 7 — Extension / Plugin Interface

python

```
class MeshExtension(Protocol):
    def pre_verify(self, payload: dict) -> None: ...
    def post_verify(self, payload: dict, result: VerifyResult) -> None: ...
    def pre_settle(self, payload: dict) -> Optional[dict]: ...   # may attach d
    def post_settle(self, receipt: dict) -> None: ...
    def enrich_receipt(self, receipt: dict) -> dict: ...          # may add fie
```

Demo extension: Bazaar discovery registration on `post_settle`. Core amount/recipient fields are passed by value into every hook, so an extension physically cannot rewrite payment terms the client signed for — enforcing "does not silently change core payment semantics" as a mechanism, not a policy statement.

---

8. Efficiency Analysis

Property	Ethereum L1	Base (L2) / Solana	Algorand
Fee per transaction	~$0.50–5+	fractions of a cent	fractions of a cent
Soft-confirmation speed	~12s/block	~1–2s	~2.9s
True finality model	probabilistic, minutes	inherits L1 finality lag (Base) / brief pre-finalization fork risk (Solana)	deterministic, single-round, no reorg after certification
Atomic payment+receipt, no deployed contract	needs a contract, gas per write	needs a program/contract	native atomic transaction groups
Free structured on-chain metadata	needs event logs (gas)	needs account writes	Note field, ~1KB, effectively free

Honest framing: fee/speed alone no longer differentiates Algorand from Base or Solana. What does, specifically for a settlement mesh, is that Modules 4 and 5's correctness guarantees come from base-layer primitives rather than a deployed, audited, called contract — a smaller attack surface and a shorter, defensible state machine.

9. Threat Model, Trust Assumptions, and Safety-Boundary Compliance

9.1 Threat model

Threat	Mitigation	Module
Duplicate settlement, pre-signing retry	Layer A idempotency key	4
Duplicate settlement, replayed signed payload	Layer B TxID claim	4
Facilitator lies about settlement outcome	Independently signed, separately-keyed receipt; on-chain Note field	5
Facilitator becomes single point of trust	Two-facilitator mesh + circuit breaker; roadmap: hash-chain audit anchor	2
Payload manipulation	Exact-match verification	3
Expired payload replay	Explicit expiry check pre-settlement	3
Network mislabeling	Compatibility layer rejects undeclared pairs	1
Facilitator outage mid-settlement	Circuit breaker, explicit `503`, no silent retry loop	2
Retry storm under network delay	`TIMEOUT` distinct from `FAILED`; idempotency key persists	4
Reorg (documented, not faked)	Out of scope post-certification per Algorand's deterministic finality; tested equivalent is facilitator crash mid-settlement	4
Private-key custody by the mesh	Never occurs — mesh only handles signed payloads	all

9.2 Trust assumptions (stated explicitly, per PS requirement — not left implicit)

1. Algorand consensus finality is trusted as authoritative once a block is certified; the mesh does not independently re-derive consensus.
2. The facilitator's Ed25519 receipt-signing key is assumed uncompromised; key rotation and HSM-backed storage are named as production hardening, not implemented in the hackathon build.
3. Redis is trusted as a private, single-tenant store not exposed outside the mesh's internal network — it is not a public-facing component and carries no independent

authentication in this build.

4. The resource server is trusted to report fulfillment honestly via the Module 6 webhook; the mesh does not verify that the resource was actually served, only that the resource server claimed it was.
5. LocalNet's cloned ledger state is trusted as behaviorally equivalent to TestNet/MainNet consensus for the properties tested here (atomicity, finality latency); it is explicitly not claimed equivalent for network-level properties like real relay propagation delay.

9.3 Safety-boundary compliance (verbatim PS requirements, addressed directly)

- **Non-custodial:** the mesh never holds, requests, or stores private keys or seed phrases at any point in any module — only signed payloads and, for the fee-payer extension case, a facilitator-controlled fee account explicitly scoped to gas sponsorship, never to moving user funds.
- **Simulated receipts are marked as simulated, not real settlement:** every receipt issued against LocalNet carries `"network":"algorand:localnet"` in both the Note-field JSON and the off-chain signed receipt — this field is never omitted or defaulted, so a receipt can never be mistaken for a MainNet settlement record by a downstream consumer.
- **Not a sanctions/fraud-evasion tool:** the mesh performs no identity verification, KYC/KYT, or compliance screening of any kind, and makes no claim to. This is stated as an explicit scope boundary rather than a silent omission — any production deployment would need a compliance layer bolted on separately, which this project does not attempt and does not claim to solve.
- **Fail closed on ambiguity:** demonstrated concretely at three points — Module 2's routing (no healthy facilitator → `503`, not a silent retry), Module 3's verification (any unresolved check → reject, not default-accept), Module 4's timeout handling (`TIMEOUT` is a distinct non-success state, never silently promoted to `CONFIRMED`).

---

10. Conformance Matrix, Official-Baseline Honesty, and Benchmarking Methodology

10.1 Conformance matrix

Scheme	Network	Status	Notes
exact	algorand:localnet	✅ Tested	Primary demo path
exact	algorand:testnet	⬜ Not tested	Named as future work, not claimed
deferred	any	⬜ Not implemented	Out of scope, not claimed

This table is generated from the same `SUPPORTED_PAIRS` set Module 1 actually enforces — the claim cannot drift from the implementation.

10.2 Official baseline — stated honestly

No publicly available, AVM-targeted, protocol-official conformance test suite was found. The closest existing artifact is `HexHive/x402scope`, a security-testing tool scoped to EVM/Solana facilitators — not directly runnable against an AVM facilitator and not an official Coinbase/x402-foundation deliverable. Given that, this report does not claim to have run against an official suite. Instead, the organizer-equivalent baseline used is the set of invariants stated directly in the `x402-foundation/x402` specification itself — trust-minimization (a scheme must never let the facilitator or resource server move funds outside client intent) and explicit scheme/network pairing — each encoded as an automated test in this project's own suite (§10.3, tests 1 and 4). This is a narrower claim than "we passed the official conformance suite," and it is reported as such rather than implied otherwise.

10.3 Cross-network routing — scoped honestly

With exactly one supported `(scheme, network)` pair in this build, "routing between networks" cannot be demonstrated as actual multi-network routing — that would require a second live network integration, out of scope for this hackathon window. The honest, tested version of this requirement is: **the compatibility layer correctly rejects any request for an undeclared network**, proven by a test that submits a well-formed payload against `algorand:testnet` (unsupported in this build) and asserts a clean `unsupported_scheme_network_pair` rejection rather than a crash, timeout, or false accept. This is stated plainly rather than let the conformance matrix imply broader routing than exists.

10.4 Benchmarking methodology

Metric	Method
False rejection rate	Construct a labeled corpus: N known-valid payloads (correct signature, amount, recipient, unexpired) and N known-invalid payloads (each with exactly one field corrupted). Run all 2N through Module 3. False rejection rate = valid payloads rejected ÷ N. Reported as a single percentage with the corpus size stated.
Failover time	Trigger Facilitator B's failure injector to force 5 consecutive failures (tripping its circuit breaker), timestamp the trip, timestamp the next successful settlement via Facilitator A, report the delta in milliseconds across 10 repeated trials (mean ± stddev, not a single cherry-picked run).
Receipt completeness	For every settled payment in a test run, assert programmatically that both the on-chain Note-field receipt and the off-chain signed receipt are present, parse correctly, and agree on `txid`/`amount`/`status`. Completeness = agreeing pairs ÷ total settlements.
Duplicate/replay rejection rate	Fire the concurrent-duplicate test (Module 4, §11 step 3) across a range of concurrency levels (2, 10, 50 simultaneous identical requests) and confirm exactly one `CONFIRMED` per batch at every level, not just the smallest case.

---

11. MVP Demonstration Script

1. Settle through Facilitator A — show success and full trace including the fulfillment webhook firing.
2. Route to Facilitator B under injected failure — show circuit breaker trip after 5 failures (timed, per §10.4), automatic reroute to Facilitator A.
3. Submit identical payload 2, 10, and 50 times concurrently — show exactly one `CONFIRMED` at each concurrency level, rest `409`.
4. Simulate confirmation delay, facilitator outage, and a malformed payload — show three distinct, correct terminal states in the trace console.
5. Pull the signed off-chain receipt, verify it with an independent script against the facilitator's public key — show it validates without touching the dashboard.
6. Query the reconciliation console by `idem_key` — show the complete 402 → verify → settle → receipt → fulfillment chain in one timeline.
7. Submit a well-formed payload against an unsupported network — show the clean, explicit rejection from §10.3.

---

12. Phased Build Order

Phase	Contents	Unlocks
1	Environment, LocalNet, Module 1	Baseline
2	Module 3 verification	Correct rejection behavior
3	Module 4 idempotency, state machine, confirmation tracking	Core correctness claim
4	Module 2 registry + second facilitator with failure injector	MVD, facilitator disagreement, circuit breaker
5	Module 5 receipt service (atomic group + signed off-chain receipt)	Independently verifiable receipts
6	Module 6 trace store + fulfillment webhook + console	Full-lifecycle observability
7	Module 7 extension interface + Bazaar hook	Extension deliverable
8	§9 threat model/trust assumptions doc, §10 conformance + benchmarks, demo rehearsal	All remaining named deliverables

---

13. Deliverables Checklist

- [ ] Protocol parser/validator (Module 1)
- [ ] Facilitator registry, health/routing layer (Module 2)
- [ ] Verification API (Module 3)
- [ ] Settlement state machine — idempotency, replay protection, **confirmation tracking** (Module 4)
- [ ] Receipt service (Module 5)
- [ ] Two facilitator simulator instances (Module 2)
- [ ] Conformance tests + **stated official-baseline scope** (§10.2)
- [ ] Dashboards + **fulfillment-inclusive trace** (Module 6)
- [ ] Audit exports (Module 6)
- [ ] Threat model + **trust assumptions** (§9.1, §9.2)
- [ ] **Safety-boundary compliance statement** (§9.3)
- [ ] Failure-injection demo (§11)
- [ ] Support/conformance matrix (§10.1)
- [ ] **Benchmarking methodology + results** (§10.4)

---

14. References

1. x402-foundation (formerly Coinbase). *x402: A payments protocol for the internet.* `github.com/x402-foundation/x402`, specification v2, 2025–2026.
2. Li, Y., Wang, L., Wang, K., Yang, Z., Wang, K., Guan, Z., Gao, J. *A402: Binding Cryptocurrency Payments to Service Execution for Agentic Commerce.* arXiv:2603.01179, 2026.
3. *SoK: Blockchain Agent-to-Agent Payments.* arXiv:2604.03733, 2026.
4. Vaziry, A., Rodriguez Garzon, S., Küpper, A. *Towards multi-agent economies: Enhancing the A2A protocol with ledger-anchored identities and x402 micropayments for AI agents.* arXiv:2507.19550, 2025.
5. Xu, M. et al. *The Agent Economy: A Blockchain-Based Foundation for Autonomous AI Agents.* arXiv:2602.14219, 2026.
6. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N. *Algorand: Scaling Byzantine Agreements for Cryptocurrencies.* SOSP '17, ACM, 2017.
7. Rivest, R.L., Shamir, A. *PayWord and MicroMint: Two Simple Micropayment Schemes.* Security Protocols 1996, LNCS vol. 1189, Springer, 1997.
8. Poon, J., Dryja, T. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments.* 2015 (contrast case, not adopted).
9. GoPlausible. *x402-avm: Algorand (AVM) extension for the x402 protocol.* Reference Python/TypeScript SDK.
10. HexHive. *x402scope.* EVM/Solana x402 facilitator security-testing tool (cited for scope contrast, §2 and §10.2).

This report is scoped to what is honestly buildable and demonstrable on Algorand LocalNet within a hackathon window. State Proofs, payment-channel batching, and multi-chain routing beyond `algorand:localnet` *are named as roadmap items because they are not claimed as delivered — see §10.1's conformance matrix and §10.3's routing-scope statement.*